

Bike Sharing App

Requirement Analysis: Use-Case Analysis

Nhóm dự án ứng dụng thuê xe đạp

Tháng 5 năm 2026

Tài liệu được điền theo template Use Case Specification của buổi học Requirement Analysis.

Mục lục

1	Phạm vi phân tích use case	2
1.1	Hệ thống và actor	2
1.2	Quy tắc nghiệp vụ chính	2
1.3	Năm use case quan trọng	2
1.4	Phân rã use case chính thành use case con	3
1.5	Ghi chú về Customer, Admin và Maintenance Staff	4
1.6	Use case overview	4
2	Relational schema	5
2.1	Các quan hệ chính	5
2.2	Quan hệ giữa các bảng	6
2.3	Ràng buộc nghiệp vụ quan trọng	6
3	Detailed use-case specifications	8
3.1	Use Case “Register and Verify Account”	8
3.2	Use Case “Manage Wallet and Rental Packages”	12
3.3	Use Case “Rent Bike by Hour”	15
3.4	Use Case “End Ride and Settle Charges”	19
3.5	Use Case “Resolve Bike Maintenance Request”	23

1 Phạm vi phân tích use case

1.1 Hệ thống và actor

Hệ thống được phân tích là **Bike Sharing System**: ứng dụng thuê xe đạp theo mô hình gần với các dịch vụ như TNGO. Người dùng cần tài khoản đã xác thực, nạp tiền vào ví hoặc mua gói ngày/tháng, thuê xe theo giờ, kết thúc chuyến đi để hệ thống tính tiền và trừ ví. Nếu người dùng vi phạm quy tắc sử dụng, hệ thống ghi nhận khoản phạt và trừ vào ví theo quy định.

Trong sơ đồ use case, front-end, back-end và database không được xem là actor vì chúng là thành phần kỹ thuật bên trong hệ thống.

Actor	Vai trò trong hệ thống
Customer	Người dùng thuê xe: đăng ký/đăng nhập có xác thực, nạp ví, mua gói ngày/tháng, thuê xe, kết thúc chuyến, nhận hóa đơn và có thể báo cáo sự cố xe.
Admin / Operator	Nhân sự vận hành: quản lý xe/trạm, chính sách giá, quy tắc sử dụng, mức phạt, phân loại sự cố và phân công bảo trì.
Maintenance Staff	Nhân sự bảo trì: nhận việc kiểm tra xe, sửa xe, cập nhật kết quả bảo trì và trạng thái sẵn sàng của xe.
GPS / Map Service	Dịch vụ bản đồ/vị trí bên ngoài hỗ trợ định vị người dùng, xe và trạm hợp lệ.
Payment Gateway	Cổng thanh toán bên ngoài xử lý giao dịch nạp ví hoặc mua gói.

1.2 Quy tắc nghiệp vụ chính

- Người dùng chỉ được thuê xe khi tài khoản đã xác thực, không bị khóa và không có chuyến đang chạy.
- Người dùng cần có số dư ví tối thiểu hoặc gói ngày/tháng còn hiệu lực trước khi bắt đầu thuê.
- Chuyến đi được tính theo thời gian thuê. Gói ngày/tháng có thể miễn hoặc giảm phí sử dụng trong phạm vi gói, nhưng không miễn các khoản phạt.
- Khi kết thúc chuyến, hệ thống kiểm tra nơi trả xe, trạng thái khóa xe, thời lượng thuê và quy tắc sử dụng để tính phí cuối cùng.
- Vi phạm như trả sai khu vực, không khóa xe, quá thời gian gói, làm hỏng/mất xe hoặc có khoản nợ chưa thanh toán sẽ phát sinh penalty.

1.3 Năm use case quan trọng

Nhóm chọn 5 use case theo tiêu chí: bao phủ luồng thuê xe trả trước, phản ánh mô hình ví/gói, có đủ vai trò người dùng và vận hành, đồng thời mỗi use case là một mục tiêu nghiệp vụ hoàn chỉnh chứ không phải một bước thao tác nhỏ.

Mã	Use case	Actor chính	Lý do chọn
UC001	Register and Verify Account	Customer	Mượn xe là tài sản có giá trị, nên tài khoản phải được xác thực chặt.
UC002	Manage Wallet and Rental Packages	Customer, Payment Gateway	Người dùng nạp ví hoặc mua gói trước khi thuê xe.
UC003	Rent Bike by Hour	Customer, GPS / Map Service	Luồng cốt lõi: tìm xe, kiểm tra điều kiện thuê và mở khóa xe.
UC004	End Ride and Settle Charges	Customer	Kết thúc chuyến, tính phí theo giờ/gói, trừ ví và ghi nhận phạt nếu có.
UC005	Resolve Bike Maintenance Request	Admin / Operator, Maintenance Staff	Customer chỉ báo cáo sự cố; xử lý bảo trì là nghiệp vụ vận hành nội bộ.

1.4 Phân rã use case chính thành use case con

Bảng dưới đây giúp phần đặc tả không dừng ở mức quá tổng quát. Các use case con vẫn là mục tiêu nghiệp vụ có ý nghĩa với actor, không phải các bước UI như “bấm nút”, “nhập ô”, hay thao tác nội bộ của backend.

UC chính	Tên UC chính	UC con / related use cases cần xét trong đặc tả
UC001	Register and Verify Account	Register Customer Account; Verify Contact Method; Submit Identity Verification; Log In Securely; Recover Account Access.
UC002	Manage Wallet and Rental Packages	Top Up Wallet; Buy Day Pass; Buy Monthly Pass; View Wallet and Package History; Retry or Cancel Pending Payment.
UC003	Rent Bike by Hour	Search Available Bikes; Scan Bike QR Code; Start Hourly Ride; Apply Active Package; View Active Ride.
UC004	End Ride and Settle Charges	Return Bike; Lock and End Ride; Report Bike Issue; Settle Ride Charge; View Receipt; Pay Outstanding Balance; Dispute Penalty.

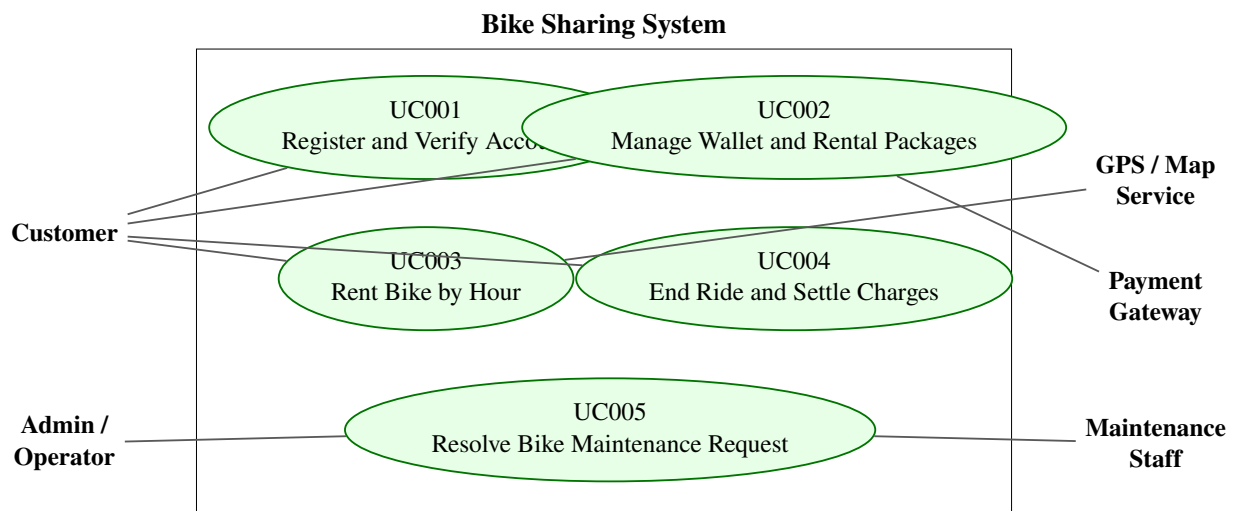
UC005	Resolve Bike Maintenance Request	Review Issue Report; Create or Merge Maintenance Request; Assign Maintenance Task; Update Repair Result; Release or Retire Bike.
-------	----------------------------------	--

1.5 Ghi chú về Customer, Admin và Maintenance Staff

Customer không phải actor trực tiếp của **UC005 Resolve Bike Maintenance Request**. Customer có thể báo cáo lỗi xe trong quá trình kết thúc chuyến hoặc qua chức năng báo cáo sự cố; báo cáo đó là dữ liệu đầu vào để Admin / Operator phân loại và tạo yêu cầu bảo trì. Sau đó Admin / Operator và Maintenance Staff cùng tham gia UC005 để xử lý xe lỗi.

Admin / Operator và Maintenance Staff có điểm giao nhau ở UC005: Admin / Operator tiếp nhận, phân loại, khóa trạng thái xe và phân công; Maintenance Staff kiểm tra, sửa chữa và cập nhật kết quả. Hai actor này cùng phục vụ mục tiêu nghiệp vụ là đưa xe lỗi trở lại trạng thái an toàn hoặc xác nhận xe không thể tiếp tục khai thác.

1.6 Use case overview



Lưu ý trình bày: sơ đồ chi tiết được lưu trong file drawio kèm theo gồm 6 trang: *UC Overview*, *UC001 Register and Verify Account*, *UC002 Manage Wallet and Rental Packages*, *UC003 Rent Bike by Hour*, *UC004 End Ride and Settle Charges*, và *UC005 Resolve Bike Maintenance Request*.

2 Relational schema

2.1 Các quan hệ chính

Lược đồ quan hệ dưới đây tập trung vào dữ liệu cần thiết cho 5 use case đã chọn. Ký hiệu: **PK** là khóa chính, **FK** là khóa ngoại.

Bảng	Thuộc tính chính
USERS	<u>user_id</u> (PK), full_name, phone, email, password_hash, role, status, created_at, last_login_at
CUSTOMER_PROFILES	<u>customer_id</u> (PK, FK → USERS.user_id), identity_number, identity_status, risk_level, verified_at
VERIFICATION_REQUESTS	<u>request_id</u> (PK), customer_id (FK → CUSTOMER_PROFILES.customer_id), method, submitted_data_ref, verification_status, reviewed_by (FK → STAFF.staff_id, nullable), rejected_reason, created_at, reviewed_at
WALLETS	<u>wallet_id</u> (PK), customer_id (FK → CUSTOMER_PROFILES.customer_id), balance, hold_amount, wallet_status, updated_at
WALLET_TRANSACTIONS	<u>transaction_id</u> (PK), wallet_id (FK), type, amount, reference_type, reference_id, status, created_at
RENTAL_PACKAGES	<u>package_id</u> (PK), package_name, package_type, price, valid_days, included_minutes, overtime_rate, active
CUSTOMER_PACKAGES	<u>customer_package_id</u> (PK), customer_id (FK), package_id (FK), starts_at, expires_at, remaining_minutes, status
STATIONS	<u>station_id</u> (PK), station_name, address, latitude, longitude, capacity, status
BIKES	<u>bike_id</u> (PK), station_id (FK → STATIONS.station_id), bike_code, bike_type, bike_status, last_maintenance_at
BIKE_LOCKS	<u>lock_id</u> (PK), bike_id (FK → BIKES.bike_id), lock_status, firmware_version, last_sync_at
RIDES	<u>ride_id</u> (PK), customer_id (FK), bike_id (FK), start_station_id (FK), end_station_id (FK), customer_package_id (FK, nullable), started_at, ended_at, ride_status, duration_minutes
RIDE_CHARGES	<u>charge_id</u> (PK), ride_id (FK → RIDES.ride_id), base_fee, package_discount, overtime_fee, penalty_amount, total_amount, settled_at
PAYMENTS	<u>payment_id</u> (PK), transaction_id (FK → WALLET_TRANSACTIONS.transaction_id), gateway_transaction_id, payment_method, amount, payment_status, paid_at
PENALTY_RULES	<u>rule_id</u> (PK), rule_name, trigger_condition, penalty_amount, active

PENALTIES	<u>penalty_id</u> (PK), ride_id (FK), rule_id (FK), amount, reason, status, created_at
ISSUE_REPORTS	<u>report_id</u> (PK), bike_id (FK), reported_by (FK → USERS.user_id), ride_id (FK, nullable), issue_description, report_status, created_at
STAFF	<u>staff_id</u> (PK, FK → USERS.user_id), staff_code, staff_role, assigned_area, active
MAINTENANCE_REQUESTS	<u>request_id</u> (PK), report_id (FK → ISSUE_REPORTS.report_id, nullable), bike_id (FK), assigned_to (FK → STAFF.staff_id), priority, request_status, repair_note, created_at, resolved_at

2.2 Quan hệ giữa các bảng

- Một USERS có thể là một CUSTOMER_PROFILES hoặc một STAFF tùy theo role.
- Một CUSTOMER_PROFILES có thể có nhiều VERIFICATION_REQUESTS theo thời gian; request gần nhất quyết định trạng thái xác thực hiện tại nếu quy trình cần duyệt lại.
- Một CUSTOMER_PROFILES có đúng một WALLETS và có nhiều WALLET_TRANSACTIONS.
- Một giao dịch nạp ví hoặc mua gói có thể được đối soát qua PAYMENTS và Payment Gateway.
- Một CUSTOMER_PROFILES có thể sở hữu nhiều CUSTOMER_PACKAGES theo thời gian; chỉ các gói còn hiệu lực mới được dùng khi thuê.
- Một CUSTOMER_PROFILES có nhiều RIDES; mỗi RIDE dùng đúng một BIKE và có tối đa một CUSTOMER_PACKAGES áp dụng.
- Mỗi RIDE có một RIDE_CHARGES khi kết thúc chuyến; tổng tiền được trừ qua WALLET_TRANSACTION.
- Một RIDE có thể phát sinh nhiều PENALTIES dựa trên PENALTY_RULES.
- Customer có thể tạo ISSUE_REPORTS, nhưng MAINTENANCE_REQUESTS là luồng xử lý của Admin / Operator và Maintenance Staff.

2.3 Ràng buộc nghiệp vụ quan trọng

- Không cho phép thuê xe nếu identity_status chưa được xác thực hoặc wallet_status không hoạt động.
- Tại một thời điểm, một Customer chỉ được có một RIDE ở trạng thái *in_progress*.
- Không cho phép UC003 nếu BIKES.bike_status khác *available* hoặc BIKE_LOCKS.lock_status không sẵn sàng.
- UC003 phải giữ một khoản tiền tối thiểu trong ví hoặc xác nhận gói ngày/tháng còn hiệu lực trước khi mở khóa xe.

- UC004 tính phí theo thời lượng thuê, áp dụng gói nếu hợp lệ, cộng overtime fee và penalty nếu vi phạm.
- Penalty không được miễn bởi gói ngày/tháng; nếu ví không đủ tiền, tài khoản chuyển sang trạng thái nợ hoặc bị tạm khóa thuê xe.
- Khi ISSUE_REPORTS hoặc MAINTENANCE_REQUESTS cho thấy xe không an toàn, BIKES.bike_status phải chuyển sang *maintenance*.

3 Detailed use-case specifications

3.1 Use Case “Register and Verify Account”

1. Use case code: UC001

2. Brief Description: This use case describes the interaction between Customer and Bike Sharing System when the customer wishes to register, verify identity and access bike rental services. Because the bike is a valuable asset, the system must verify account ownership and customer identity before allowing rental.

3. Actors

- Customer

4. Included / related sub-use cases

Sub UC	Name	Goal
UC001.1	Register Customer Account	Customer creates an account with phone/email and password.
UC001.2	Verify Contact Method	Customer proves ownership of phone/email by OTP.
UC001.3	Submit Identity Verification	Customer submits identity data required before renting valuable assets.
UC001.4	Log In Securely	Customer accesses an existing verified account.
UC001.5	Recover Account Access	Customer resets password or recovers account using verified contact method.
Related	Review Customer Verification	Admin / Operator may manually review suspicious or failed identity requests; this is a separate operational use case, not the main UC001 actor flow.

5. Preconditions

- Customer has internet access and a reachable phone number or email.
- The system can send OTP messages through the configured provider.
- Identity verification policy and minimum rental eligibility rules are configured.

6. Basic Flow of Events

1. Customer opens the application and chooses to register or log in.
2. The software displays account access options and the verification requirement for bike rental.
3. Customer enters account credentials and contact information (see Table A1).

4. The software validates required fields, password strength and duplicate phone/email.
5. The software sends an OTP to the selected contact method.
6. Customer submits the OTP.
7. The software verifies the OTP and creates or authenticates the account.
8. Customer submits identity information and required identity evidence.
9. The software validates the identity submission and determines identity status.
10. The software displays access status, verification status and rental eligibility (see Table B1).

7. Alternative flows

Bảng 6: Alternative flows of events for UC001

No	Location	Condition	Action	Resume
1	Step 3	Required account fields are missing or have invalid format.	The software rejects the submission, explains field-level errors and keeps the account uncreated.	Step 3
2	Step 4	Phone number or email already belongs to another account.	The software blocks duplicate registration and offers login or account recovery.	Step 1
3	Step 4	Password does not satisfy security policy.	The software displays password rules and asks Customer to submit a stronger password.	Step 3
4	Step 5	OTP provider cannot deliver the code.	The software allows resend after cooldown and records OTP delivery failure for monitoring.	Step 5
5	Step 6	OTP is incorrect, expired or submitted too many times.	The software rejects OTP; after the retry limit, it temporarily locks verification attempts.	Step 5 or Ends
6	Step 7	Login is from a new or risky device.	The software requires step-up OTP verification before allowing access.	Step 5
7	Step 8	Identity evidence is unreadable or incomplete.	The software marks identity request as rejected and asks Customer to resubmit evidence.	Step 8
8	Step 9	Identity data mismatches account data or blacklist/risk policy.	The software marks identity request as rejected or pending manual review and blocks rental.	Ends limited

9	Step 9	Verification requires manual review.	The software marks account as pending verification; Customer may use non-rental features but cannot rent bikes.	Ends pending
10	Step 10	Customer has unpaid debt or severe violation history.	The software allows login but marks rental eligibility as blocked until debt/violation is resolved.	Ends blocked

8. Input data

Bảng 7: Table A1 - Input data of account registration and verification

No	Data fields	Description	Mandatory	Valid condition	Example
1	FullName	Customer legal or registered name.	Yes	2–100 characters.	Nguyen Van A
2	PhoneNumber	Primary phone number for account and OTP.	Yes	Valid, unique, reachable.	0912345678
3	Email	Optional email for receipt and recovery.	No	Valid format if provided.	user@gmail.com
4	Password	Secret credential for login.	Yes	Meets password policy.	Abc12345!
5	OTP	Verification code sent by system.	Yes	Correct, unexpired, within retry limit.	123456
6	IdentityNumber	Citizen ID, passport, student ID or equivalent.	Yes before rental	Valid format and not blacklisted.	001203000001
7	IdentityEvidence	Identity document/selfie evidence reference.	Yes before rental	Clear image, accepted type, not reused suspiciously.	id-front.jpg

9. Output data

Bảng 8: Table B1 - Output data of account registration and verification

No	Data fields	Description	Display format	Example
1	UserID	Unique ID of the customer account.	String	U001
2	AccessStatus	Login or registration result.	String	Authenticated
3	IdentityStatus	Verification result.	String	Verified
4	RentalEligibility	Whether Customer can start rental.	Text/Boolean	Eligible
5	BlockReason	Reason if rental is blocked.	Text	Unpaid debt

10. Acceptance rules

- A customer cannot start UC003 unless IdentityStatus is *Verified*.
- OTP must expire and must have a retry limit.
- Duplicate phone numbers are not accepted for active customer accounts.
- A locked/suspended account may log in only if policy allows, but cannot rent bikes.

3.2 Use Case “Manage Wallet and Rental Packages”

1. Use case code: UC002

2. Brief Description: This use case describes the interaction between Customer, Payment Gateway and Bike Sharing System when the customer wishes to top up the prepaid wallet, buy a day/month package, or review payment/package status before renting.

3. Actors

- Customer
- Payment Gateway

4. Included / related sub-use cases

Sub UC	Name	Goal
UC002.1	Top Up Wallet	Customer adds prepaid balance to wallet.
UC002.2	Buy Day Pass	Customer buys a short-term rental package.
UC002.3	Buy Monthly Pass	Customer buys a longer-term package for repeated usage.
UC002.4	View Wallet and Package History	Customer reviews balance, package validity and transaction history.
UC002.5	Retry or Cancel Pending Payment	Customer handles a payment that has no final result yet.

5. Preconditions

- Customer is logged in with an active account.
- Customer wallet exists and is not locked.
- Payment Gateway is available or pending payment policy is configured.
- Active package catalog, prices and validity rules are configured.

6. Basic Flow of Events

1. Customer opens Wallet and Packages.
2. The software displays wallet balance, held amount, active packages and available packages.
3. Customer selects Top Up Wallet, Buy Day Pass, Buy Monthly Pass or View History.
4. For payment actions, Customer enters amount or selects package and payment method (see Table A2).
5. The software validates wallet status, amount limits, package availability and customer status.

6. The software creates a pending wallet transaction.
7. The software sends payment request to Payment Gateway.
8. Payment Gateway returns payment result.
9. The software updates wallet balance or activates the package.
10. The software displays transaction receipt, updated wallet and package status (see Table B2).

7. Alternative flows

Bảng 10: Alternative flows of events for UC002

No	Location	Condition	Action	Resume
1	Step 2	Wallet is locked due to fraud, debt or admin action.	The software displays lock reason and blocks new top-up/package purchase if policy requires.	Ends
2	Step 4	Top-up amount is outside allowed limits.	The software rejects the amount and shows minimum/maximum rules.	Step 4
3	Step 4	Selected package is expired, hidden or inactive.	The software refreshes package catalog and asks Customer to choose another package.	Step 3
4	Step 5	Customer is unverified.	The software allows balance top-up but marks rental/package usage as blocked until verification is completed.	Ends limited
5	Step 7	Payment Gateway times out before final result.	The software keeps transaction pending and prevents duplicate credit until reconciliation completes.	Step 10 pending
6	Step 8	Payment is declined.	The software marks transaction failed and allows retry with another method.	Step 4
7	Step 8	Gateway returns success but callback is duplicated.	The software ignores duplicate callback using gateway transaction ID and does not credit twice.	Step 10
8	Step 9	Customer already has an active package of same type.	The software applies configured rule: extend validity, queue next package or reject purchase.	Step 10
9	Step 9	Wallet update fails after successful payment.	The software marks transaction for reconciliation and does not show balance until fixed.	Ends pending

10	Step 10	Customer requests transaction history.	The software displays filtered wallet/package history without starting a payment.	Ends
----	---------	--	---	------

8. Input data

Bảng 11: Table A2 - Input data of wallet and packages

No	Data fields	Description	Mandatory	Valid condition	Example
1	CustomerID	Customer who performs transaction.	Yes	Existing active customer.	U001
2	TransactionType	Wallet/package action.	Yes	TopUp, BuyDayPass, BuyMonthlyPass, ViewHistory.	TopUp
3	Amount	Money paid by Customer.	Yes for payment	Positive amount within policy.	100000
4	PackageID	Package selected by Customer.	Yes for package	Existing active package.	MONTHLY01
5	PaymentMethod	Gateway payment method.	Yes for payment	Supported method.	BankCard
6	IdempotencyKey	Unique client transaction key.	Yes	Unique for request retry.	TX-abc-01

9. Output data

Bảng 12: Table B2 - Output data of wallet and packages

No	Data fields	Description	Display format	Example
1	TransactionID	Unique wallet transaction.	String	WT9001
2	WalletBalance	Available wallet balance.	Currency	85000 VND
3	HoldAmount	Balance temporarily reserved for active ride.	Currency	20000 VND
4	PackageStatus	Current package status.	String	Active
5	PaymentStatus	Gateway/payment status.	String	Paid
6	ValidUntil	Package expiry time if applicable.	Datetime	2026-06-12

10. Acceptance rules

- Successful payment must create exactly one successful wallet transaction.
- Package validity and remaining minutes must be visible before Customer starts a ride.
- Pending payment must not increase wallet balance until confirmed.
- Penalty debt cannot be hidden by buying a package.

3.3 Use Case “Rent Bike by Hour”

1. Use case code: UC003

2. Brief Description: This use case describes the interaction between Customer, GPS / Map Service and Bike Sharing System when the customer wishes to find an available bike, satisfy rental eligibility rules and start an hourly ride.

3. Actors

- Customer
- GPS / Map Service

4. Included / related sub-use cases

Sub UC	Name	Goal
UC003.1	Search Available Bikes	Customer finds rentable bikes or stations nearby.
UC003.2	Scan Bike QR Code	Customer identifies a specific bike to rent.
UC003.3	Start Hourly Ride	Customer starts a ride after eligibility and bike checks pass.
UC003.4	Apply Active Package	Customer uses a valid day/month package for the ride if applicable.
UC003.5	View Active Ride	Customer monitors active ride time, bike and charging rule.

5. Preconditions

- Customer is logged in, verified and not suspended.
- Customer has no active ride and no blocking unpaid penalty.
- Customer has sufficient wallet balance/hold amount or an active package.
- Bike, station and lock status are synchronized enough to make a rental decision.

6. Basic Flow of Events

1. Customer opens map, nearby bikes or scan screen.
2. The software requests current location from GPS / Map Service.
3. GPS / Map Service returns location or selected search area.
4. The software displays available bikes/stations and important rental rules.
5. Customer selects a bike or scans the bike QR code (see Table A3).
6. The software verifies account, wallet/package, active ride, debt, bike status and lock status.

7. The software displays hourly price, package coverage, wallet hold and penalty rules.
8. Customer accepts rental conditions.
9. The software creates ride record and reserves required wallet hold or package usage.
10. The software sends unlock command and receives successful lock response.
11. The software marks ride as in progress and bike as in use.
12. The software displays active ride details (see Table B3).

7. Alternative flows

Bảng 14: Alternative flows of events for UC003

No	Location	Condition	Action	Resume
1	Step 2	Customer denies location permission.	The software allows manual station search or QR scan without current location.	Step 4
2	Step 4	No available bike is found nearby.	The software suggests larger radius, another station or later retry.	Step 4
3	Step 5	QR code is invalid, damaged or belongs to another system.	The software rejects the scan and asks Customer to scan another bike.	Step 5
4	Step 6	Account is not verified, suspended or has unpaid debt.	The software blocks rental and shows the exact reason and required recovery action.	Ends
5	Step 6	Customer already has an active ride.	The software opens the active ride screen instead of creating another ride.	Ends
6	Step 6	Wallet balance is below required hold and no active package is available.	The software redirects Customer to top up wallet or buy a package.	UC002
7	Step 6	Bike becomes unavailable after map display.	The software rejects rental and refreshes availability.	Step 4
8	Step 6	Bike is under maintenance or lock status is offline.	The software blocks rental, hides the bike from available list and may create inspection signal.	Step 4
9	Step 8	Customer refuses rental rules or price.	The software cancels rental attempt without wallet hold.	Ends
10	Step 9	Ride record is created but wallet hold fails.	The software cancels pending ride and keeps bike available.	Ends

11	Step 10	Unlock command fails or times out.	The software retries according to policy, then cancels ride, releases hold and marks bike for inspection.	Ends
12	Step 12	App loses network after unlock succeeds.	The software keeps ride in progress on server and shows active ride when Customer reconnects.	Ends active

8. Input data

Bảng 15: Table A3 - Input data of hourly bike rental

No	Data fields	Description	Mandatory	Valid condition	Example
1	CustomerID	Customer who starts rental.	Yes	Verified active customer.	U001
2	CurrentLocation	Customer location.	No	Valid latitude/longitude if provided.	21.005,105.843
3	BikeID	Selected/scanned bike.	Yes	Existing available bike.	B102
4	StationID	Current station/area of the bike.	Yes	Active station/allowed area.	S01
5	RentalMode	Charging mode.	Yes	Hourly, DayPass, MonthlyPass.	Hourly
6	AcceptedRuleVersion	Rental rules accepted by Customer.	Yes	Current active version.	RULE2026A

9. Output data

Bảng 16: Table B3 - Output data of hourly bike rental

No	Data fields	Description	Display format	Example
1	RideID	Unique active ride ID.	String	R5001
2	RideStatus	Current ride status.	String	In progress
3	BikeStatus	Current bike status.	String	In use
4	StartedAt	Rental start time.	Datetime	2026-05-12 08:30
5	WalletHold	Reserved wallet balance.	Currency	20000 VND
6	AppliedPackage	Package applied if any.	String	MONTHLY01
7	ActiveRuleVersion	Rule version applied to ride.	String	RULE2026A

10. Acceptance rules

- The system must not create two active rides for one customer.

- Bike status must change from *available* to *in_use* only after successful ride start.
- Wallet hold/package reservation must be released if unlock fails.
- Customer must see the applicable price/package/penalty rules before confirmation.

3.4 Use Case “End Ride and Settle Charges”

1. Use case code: UC004

2. Brief Description: This use case describes the interaction between Customer and Bike Sharing System when the customer wishes to end a ride. The system checks return rules, calculates hourly fee, package coverage, overtime and penalties, then deducts the final amount from the prepaid wallet.

3. Actors

- Customer

4. Included / related sub-use cases

Sub UC	Name	Goal
UC004.1	Return Bike	Customer returns bike to valid station/area.
UC004.2	Lock and End Ride	Customer locks bike and requests ride completion.
UC004.3	Report Bike Issue	Customer reports damage or safety issue discovered during/after ride.
UC004.4	Settle Ride Charge	Customer pays ride fee, overtime fee and penalties from wallet.
UC004.5	View Receipt	Customer receives receipt and wallet/package update.
UC004.6	Pay Outstanding Balance	Customer clears debt if wallet is insufficient.
UC004.7	Dispute Penalty	Customer asks Admin / Operator to review a penalty; this is related but can be handled after settlement.

5. Preconditions

- Customer has an active ride.
- Bike can be locked or lock status can be verified.
- Settlement rules, package usage and penalty rules are available.

6. Basic Flow of Events

1. Customer reaches a station or allowed return area.
2. Customer locks the bike and selects End Ride.
3. The software asks Customer to confirm return location, lock status and optional bike condition note (see Table A4).
4. The software verifies return area, lock status and ride duration.

5. The software calculates base hourly fee and package-covered minutes.
6. The software checks violation rules: wrong return, unlocked bike, overtime, damage/loss report or debt rule.
7. The software creates penalty records for detected violations.
8. The software calculates final charge and deducts wallet balance/hold.
9. The software updates ride, bike, wallet, package and issue report status.
10. The software displays receipt, updated wallet balance and penalty/issue report result (see Table B4).

7. Alternative flows

Bảng 18: Alternative flows of events for UC004

No	Location	Condition	Action	Resume
1	Step 1	Customer is outside valid return area.	The software warns Customer and shows nearest valid station/area; if Customer confirms wrong return, penalty rule is applied.	Step 3 or Step 6
2	Step 2	Bike is not locked.	The software refuses normal completion and asks Customer to lock bike; ride time continues until lock is confirmed.	Step 2
3	Step 4	Lock signal cannot be synchronized.	The software creates pending review, stores Customer evidence and may keep ride pending until Admin review.	Ends pending
4	Step 5	Active package covers entire ride.	The software sets base ride fee to zero but still checks overtime/penalty rules.	Step 6
5	Step 5	Ride exceeds package included minutes.	The software calculates overtime fee from package overtime rate or hourly policy.	Step 6
6	Step 6	Wrong return area, unlocked return, damage, loss or overdue debt is detected.	The software creates one or more penalties using PENALTY_RULES.	Step 7
7	Step 8	Wallet balance is insufficient.	The software deducts available amount, records outstanding balance and blocks new rentals until paid.	Step 9
8	Step 8	Wallet deduction fails due to concurrency or ledger error.	The software marks settlement pending and prevents duplicate deduction until reconciliation.	Ends pending

9	Step 9	Customer reports a bike issue.	The software creates ISSUE_REPORTS and marks bike as inspection required if issue affects safety.	Step 10
10	Step 9	Bike is returned normally with no issue.	The software marks bike available for next rental.	Step 10
11	Step 10	Customer disputes a penalty.	The software records a dispute request while keeping charge according to policy until review completes.	Ends
12	Step 10	Receipt generation fails.	The software completes settlement and lets Customer view receipt later from ride history.	Ends

8. Input data

Bảng 19: Table A4 - Input data of ride settlement

No	Data fields	Description	Mandatory	Valid condition	Example
1	RideID	Active ride to be completed.	Yes	Existing in-progress ride.	R5001
2	EndStationID	Return station/area.	Yes	Valid active station/area or flagged wrong return.	S03
3	LockConfirmation	Lock signal or customer confirmation.	Yes	True or pending-review evidence.	True
4	EndedAt	Ride end request time.	Yes	After StartedAt.	2026-05-12 09:05
5	BikeConditionNote	Optional issue note.	No	Text up to configured limit.	Brake issue
6	EvidenceImage	Optional photo for issue/dispute.	No	Accepted image type.	issue.jpg
7	RuleVersion	Rule version used at settlement.	Yes	Active at ride start/end per policy.	RULE2026A

9. Output data

Bảng 20: Table B4 - Output data of ride settlement

No	Data fields	Description	Display format	Example
1	ReceiptID	Settlement receipt ID.	String	RC9001

2	RideStatus	Final ride status.	String	Completed
3	RideFee	Fee after package coverage.	Currency	12000 VND
4	PenaltyAmount	Total penalty amount.	Currency	50000 VND
5	TotalCharge	Final deducted amount.	Currency	62000 VND
6	WalletBalance	Balance after settlement.	Currency	38000 VND
7	IssueReportID	Issue report if created.	String	IR3001

10. Acceptance rules

- A completed ride must have exactly one settlement result or a clear pending-review status.
- Package may reduce ride fee but must not remove penalty charges.
- Wallet deduction must be auditable in WALLET_TRANSACTIONS.
- Unsafe bikes must not become available after settlement.

3.5 Use Case “Resolve Bike Maintenance Request”

1. Use case code: UC005

2. Brief Description: This use case describes the interaction between Admin / Operator, Maintenance Staff and Bike Sharing System when a reported or detected bike issue must be reviewed, assigned, repaired and closed. Customer is not an actor of this use case; Customer only creates an issue report that can trigger this workflow.

3. Actors

- Admin / Operator
- Maintenance Staff

4. Included / related sub-use cases

Sub UC	Name	Goal
UC005.1	Review Issue Report	Admin / Operator evaluates report validity and severity.
UC005.2	Create or Merge Maintenance Request	Admin / Operator creates a request or merges duplicated reports.
UC005.3	Assign Maintenance Task	Admin / Operator assigns repair work to available staff.
UC005.4	Update Repair Result	Maintenance Staff records inspection and repair result.
UC005.5	Release or Retire Bike	Admin / Operator or Maintenance Staff marks bike available, still unsafe or retired.

5. Preconditions

- A bike exists in the system.
- An issue report, failed lock event or operational inspection indicates that the bike may need maintenance.
- Admin / Operator and Maintenance Staff are authenticated with appropriate roles.

6. Basic Flow of Events

1. Admin / Operator opens issue reports or bikes requiring inspection.
2. Admin / Operator reviews report details, ride link, evidence and bike status (see Table A5).
3. The software creates a maintenance request or links the report to an existing open request.
4. The software marks the bike as maintenance or inspection required.
5. Admin / Operator sets priority and assigns the request to Maintenance Staff.

6. Maintenance Staff receives the assigned maintenance task.
7. Maintenance Staff inspects the bike, performs repair or records that repair cannot be completed.
8. Maintenance Staff updates repair notes, parts used, final safety status and evidence.
9. The software validates required closure data and updates request status.
10. The software updates bike status and related issue reports (see Table B5).

7. Alternative flows

Bảng 22: Alternative flows of events for UC005

No	Location	Condition	Action	Resume
1	Step 2	Issue report is invalid, spam or unrelated to the bike.	Admin / Operator rejects report with reason and keeps bike status unchanged if no safety risk exists.	Ends
2	Step 3	The bike already has an open maintenance request.	The software merges the report into existing request and updates priority if needed.	Step 5
3	Step 4	Issue indicates severe safety risk.	The software immediately hides bike from rental and marks it unavailable before assignment.	Step 5
4	Step 5	No Maintenance Staff is available.	The software keeps request unassigned, notifies Admin / Operator and maintains bike as unavailable.	Step 5
5	Step 5	Assigned staff rejects task due to area/skill mismatch.	Admin / Operator reassigns task or changes priority/area.	Step 5
6	Step 7	Bike cannot be found at expected station.	Maintenance Staff marks request as bike missing; Admin / Operator starts lost-bike handling.	Ends escalated
7	Step 7	Repair requires parts or external service.	Maintenance Staff marks request waiting for parts/external repair and records expected follow-up.	Step 7
8	Step 8	Closure data is incomplete.	The software blocks closure and asks staff to add final status, repair note and evidence.	Step 8
9	Step 9	Bike fails safety check after repair.	The software keeps bike in maintenance status and escalates to Admin / Operator.	Ends unresolved

10	Step 10	Bike is repaired and passes safety check.	The software marks request closed and bike available at its current station.	Ends
----	---------	---	--	------

8. Input data

Bảng 23: Table A5 - Input data of maintenance request

No	Data fields	Description	Mandatory	Valid condition	Example
1	ReportID	Issue report that triggered maintenance.	No	Existing open report if available.	IR3001
2	BikeID	Bike affected by the issue.	Yes	Existing bike.	B102
3	Priority	Operational severity.	Yes	Low, Medium, High, Critical.	High
4	IssueDescription	Problem reviewed by Admin / Operator.	Yes	Meaningful text.	Brake loose
5	AssignedStaffID	Staff member assigned to repair.	Yes when assigned	Existing active staff.	ST05
6	RepairNote	Inspection and repair result.	Yes when closing	Meaningful text.	Adjusted brake
7	FinalBikeStatus	Status after maintenance.	Yes when closing	Available, Maintenance, Retired, Lost.	Available

9. Output data

Bảng 24: Table B5 - Output data of maintenance request

No	Data fields	Description	Display format	Example
1	RequestID	Maintenance request ID.	String	M3001
2	RequestStatus	Current/final request status.	String	Closed
3	BikeStatus	Updated bike status.	String	Available
4	AssignedStaff	Staff assigned to request.	Text	Nguyen Van B
5	ResolvedAt	Completion time.	Datetime	2026-05-12 15:20
6	LinkedReports	Issue reports resolved/merged by this request.	List	IR3001, IR3002

10. Acceptance rules

- Customer is not connected to UC005 in the use-case diagram.
- A bike with an open safety-related maintenance request must not be rentable.

- Request closure requires final bike status and repair note.
- Duplicated issue reports should be merged rather than creating many open requests for one bike.